

Vulnerability Exceptions

Exception governance, risk acceptance, compensating controls, expiry tracking, and retest closure criteria.

Field	Value
Project	Enterprise DevSecOps Security Lab
Document Owner	Jagriti Banerjee
Document Type	Vulnerability Exceptions
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, GHCR
Classification	Portfolio / Private Lab Evidence
Status	Final Detailed Spacing Fixed

Document Scope

This report is part of the Enterprise DevSecOps evidence package. It is written for technical reviewers and recruiters who want to understand how artifact trust, SBOM visibility, provenance, VEX notes, and exception governance were implemented and validated in the private lab.

Reviewer Focus Area	What this document proves
Policy	Defines what can and cannot receive a temporary exception.
Ownership	Assigns exception responsibilities, approval criteria, expiry, and evidence requirements.
Closure	Requires remediation, VEX justification, or retest proof before closing risk.

Evidence Package	Included
Supply chain controls	Cosign signing, Syft SBOM, provenance attestation, Gatekeeper metadata controls
Decision records	Security gate policy, exception rules, VEX status notes, retest expectations
Release quality focus	Artifact integrity, component transparency, vulnerability triage, and audit readiness

Executive Summary

This document defines the vulnerability exception process for the Enterprise DevSecOps project. It ensures that known vulnerabilities are not ignored silently. Every exception must be owned, justified, time-boxed, compensated, and retested. The process also distinguishes intentional lab vulnerabilities from production release candidates.

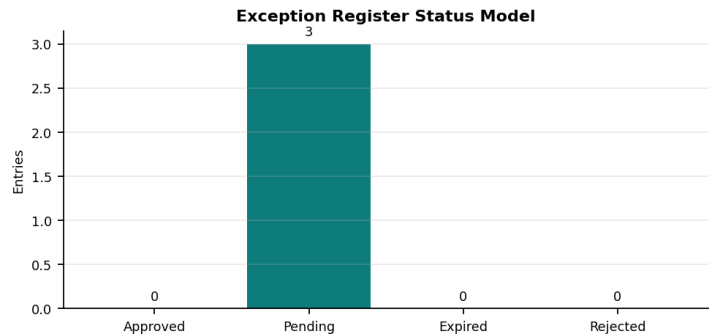


Figure 1 - Exception workflow state model.

Blocking Gate Rules

Gate Condition	Decision	Reason
Critical dependency vulnerability	Fail	Dependency issues can be exploitable before runtime hardening helps.
Critical container vulnerability	Fail	Base image or OS packages may compromise the image execution boundary.
Secret found	Fail	Secrets in source or image create immediate credential exposure risk.
Dockerfile critical misconfiguration	Fail	Build and runtime misconfiguration can weaken isolation and provenance.
Kubernetes manifest critical misconfiguration	Fail	Critical workload misconfigurations can create cluster compromise paths.

Roles and Responsibilities

Role	Responsibility	Expected Evidence
Developer	Fix dependency, code, image, or manifest issues.	Commit, test result, updated scanner output.
Security reviewer	Approve or reject exception requests.	Risk decision, reason, expiry, compensating controls.
Project owner	Accept non-production lab risk.	Signed-off exception record or closure note.
CI/CD pipeline	Detect and block critical findings.	Workflow result, SARIF, artifacts, gate summary.
Release reviewer	Confirm all exceptions are closed or valid.	Release checklist and evidence bundle.

Exception Request Template

Exception ID: EXC-YYYY-###

Finding ID: <CVE/CWE/scanner ID/report ID>

Affected Artifact: <package, image, Dockerfile, Kubernetes manifest>

Severity: <Critical/High/Medium/Low>

Request Reason: <why immediate remediation is blocked>

VEX Status: affected | not_affected | fixed | under_investigation

Compensating Controls: <runtime, network, admission, monitoring controls>

Owner: <person/team>

Approval Status: pending | approved | rejected | expired

Expiry Date: <date>

Retest Evidence Required: <scanner output, patch, SBOM, VEX note>

Example Exception Register

ID	Finding	Status	Expiry	Required Next Action
EXC-001	Intentional Flask SQL injection	Lab-only documented	N/A	Keep isolated; do not promote vulnerable code.
EXC-002	Outdated dependency from requirements.txt	Pending	30 days	Upgrade package and rerun pip-audit.
EXC-003	Container image critical CVE	Pending	14 days	Rebuild from patched base image and rerun Trivy.
EXC-004	Privileged pod pattern	Closed	Closed	Blocked by Pod Security restricted policy.

Closure and Audit Requirements

- Exception closure requires evidence, not just status text.
- Expired exceptions must be escalated or returned to remediation.
- Critical findings cannot be hidden by VEX; VEX only explains exploitability context.
- All exceptions should link to SBOM, scanner output, VEX note, and retest proof.
- Exception records should be reviewed before every release or portfolio demonstration.

References

- CISA VEX minimum requirements: <https://www.cisa.gov/sites/default/files/2023-04/minimum-requirements-for-vex-508c.pdf>
- OpenVEX specification: <https://github.com/openvex/spec/blob/main/OPENVEX-SPEC.md>
- SLSA specification: <https://slsa.dev/spec/v1.2/>
- SLSA provenance specification: <https://slsa.dev/spec/v1.0/provenance>
- Sigstore Cosign project: <https://github.com/sigstore/cosign>
- Cosign specifications and attestation support: https://docs.sigstore.dev/cosign/system_config/specifications/
- Sigstore project overview: <https://www.sigstore.dev/>